Simple Encoder-Decoder for Image Segmentation

Rad Tudor-Andrei, Radu Tudor-Eduard

1 U-Net Model

U-Net is a type of Convolutional Neural Network, specialized in image segmentation. The name comes from the architecture of the model which is resembling the letter 'U'. It is made up of:

1. Encoder:
   Uses small filters (3x3) to find features (down-sampling)
   Uses ReLU to add non-linearity
   Uses max-pooling (2x2) to shrink
2. Bottleneck:
   Middle of 'U'. Links the encoder and decoder. Holds most abstract information
3. Decoder:
   Up-sampling to get back the original image size
4. *Skip connections*:
   Helps to combine decoder details with corresponding encoder details to help with spatial information

2 Data Selection

Data is obtained from a dataset available on Kaggle. Data is split into train, test, and validation, and consists of lung segmentation masks and images.

3 Experimental setup

The model was trained for 30 epochs using a batch size of 1, on a single GPU environment provided by Kaggle. Training was implemented in TensorFlow/Keras. Validation loss was monitored throughout the training to observe potential overfitting.

4 Preprocessing

Although the dataset provided images of good quality and sufficient clarity, a denoising step was nonetheless applied to the input images to further reduce residual noise. This operation was performed exclusively on the image data, while the corresponding segmentation masks were left unchanged.
For this purpose, a Gaussian filtering approach was selected, as it offers an effective compromise between noise reduction and edge preservation, which is particularly important for maintaining accurate anatomical boundaries in lung segmentation tasks.

1. Gaussian kernel

```
def gaussian_kernel(size=5, sigma=1.0):
    x = np.linspace(-size//2, size//2, size)
    g = np.exp(-(x**2) / (2 * sigma**2))
    g = g / g.sum()
    kernel = np.outer(g, g)
    kernel = kernel[:, :, np.newaxis, np.newaxis]
    return tf.constant(kernel, dtype=tf.float32)
```

2. Gaussian blur

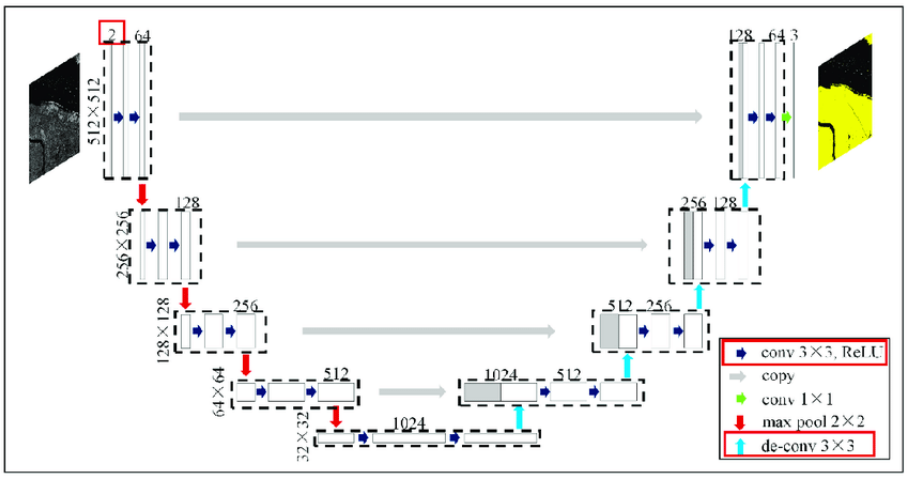
```
def gaussian_blur_tf(image, size=5, sigma=1.0):
    if len(image.shape) == 2:
        image = image[..., tf.newaxis]

    kernel = gaussian_kernel(size, sigma)
    image = tf.expand_dims(image, axis=0) # batch dim

    blurred = tf.nn.conv2d(
        image,
        kernel,
        strides=1,
        padding="SAME"
    )

    return tf.squeeze(blurred, axis=0)
```

A Gaussian filter with a kernel size of 5×5 and a standard deviation sigma = 1.0 was used to denoise the image. This configuration provides a balanced trade-off between noise reduction and edge preservation, which is essential to maintain accurate lung boundaries.
The kernel was normalized to preserve the overall intensity of the image, and convolution was performed with a unit stride and the same padding to maintain the spatial resolution and image dimensions

The application of the denoising step resulted in an improvement of approximately 2.0% in the Dice score, increasing from 0.952 to 0.967. Although this gain may appear modest, it can be attributed to the fact that the dataset already provides images of exceptionally high quality. Nevertheless, this result highlights the potential importance of denoising as a preprocessing step, particularly in scenarios involving lower-quality or noisier medical images, where its impact on segmentation performance is expected to be more pronounced.

5 Architecture

Small convolutional kernels (3×3) were chosen as they allow the network to learn fine-grained spatial features while keeping the number of parameters manageable. A sigmoid activation was used in the output layer since the task is a binary segmentation problem.



1. Encoder:

```
def encoder_block(inputs, num_filters):
    x = tf.keras.layers.Conv2D(num_filters, 3, padding = 'same')(inputs)
    x = tf.keras.layers.Activation('relu')(x)

    x = tf.keras.layers.Conv2D(num_filters, 3, padding = 'same')(x)
    x = tf.keras.layers.Activation('relu')(x)
```

```
    x = tf.keras.layers.MaxPool2D(pool_size=(2, 2), strides = 2)(x)

    return x
```

2. Decoder:

```
def decoder_block(inputs, skip_connections, num_filters):
    x = tf.keras.layers.Conv2DTranspose(num_filters, (2, 2), strides = 2, padding =
        'same')(inputs)
    skip_connections = tf.keras.layers.Resizing(x.shape[1],
        x.shape[2])(skip_connections)
    x = tf.keras.layers.Concatenate()([x, skip_connections])

    x = tf.keras.layers.Conv2D(num_filters, 3, padding = 'same')(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = tf.keras.layers.Conv2D(num_filters, 3, padding = 'same')(x)
    x = tf.keras.layers.Activation('relu')(x)

    return x
```

3. Model:

```
def unet_model(input_shape = (512, 512, 1), num_classes = 1):
    # input is 512x512 greyscale images
    # two classes: either background or lung area
    inputs = tf.keras.layers.Input(shape = input_shape)

    # Encoder
    s1 = encoder_block(inputs, 64)
    s2 = encoder_block(s1, 128)
    s3 = encoder_block(s2, 256)
    s4 = encoder_block(s3, 512)

    # Bottleneck
    b1 = tf.keras.layers.Conv2D(512, 3, padding = 'same')(s4)
    b1 = tf.keras.layers.Activation('relu')(b1)
    b1 = tf.keras.layers.Conv2D(512, 3, padding = 'same')(b1)
    b1 = tf.keras.layers.Activation('relu')(b1)

    # Decoder
    d1 = decoder_block(b1, s4, 512)
    d2 = decoder_block(d1, s3, 256)
    d3 = decoder_block(d2, s2, 128)
    d4 = decoder_block(d3, s1, 64)

    outputs = tf.keras.layers.Conv2D(num_classes, 1, padding = 'same', activation =
        'sigmoid')(d4)

    model = tf.keras.Model(inputs = inputs, outputs = outputs, name = 'U-Net')
    return model
```

6 Loss Optimization

The loss function used to punish foreground/background misclassifications is a combination of binary cross-entropy and dice-loss. Dice-loss enforces region overlap, while BCE speeds up convergence in training. Both are stable.

1. **Binary cross-entropy**:

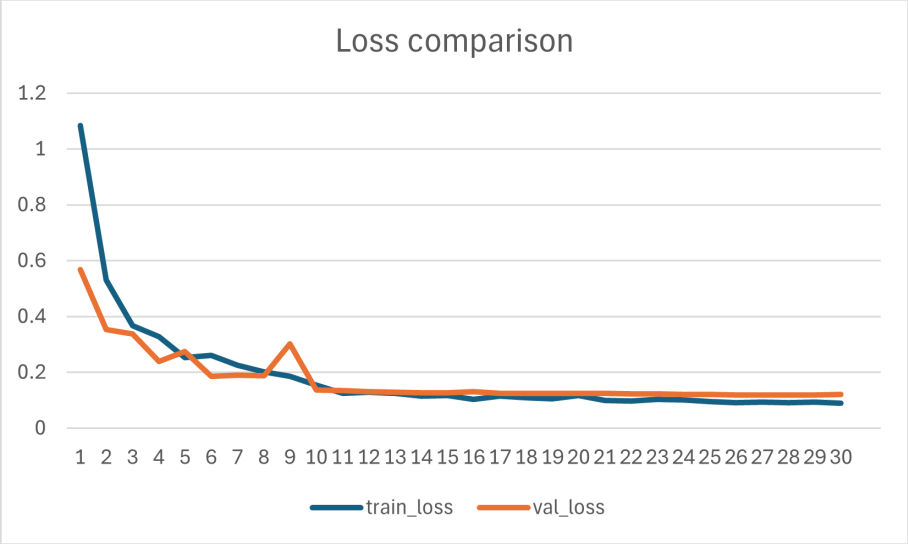
$$y_{\text{test}} \subset \{0,1\}^{H\times W}, \quad y_{\text{pred}} \subset [0,1]^{H\times W}$$

$$BCE(y, \hat{y}) = -[ylog(\hat{y}) + (1-y)log(1-\hat{y})]$$

2. **Dice-loss**

$$I = y_{test} \cap y_{pred}, \quad S = |y_{test}| + |y_{pred}|$$

$$\text{dice score} = \frac{2 \times I}{S}, \quad \text{dice loss} = 1 - \text{dice score}$$



The figure above illustrates the evolution of training and validation loss across epochs. Both curves decrease steadily and converge, indicating stable training and good generalization. The absence of a significant divergence between training and validation loss suggests that overfitting was avoided.

```
def dice_loss(y_true, y_pred, smooth=1):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(y_pred, tf.float32)
    intersection = tf.reduce_sum(y_true * y_pred)

    return 1 - (2 * intersection + smooth) / (
        tf.reduce_sum(y_true) + tf.reduce_sum(y_pred) + smooth
    )

unet.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss=lambda y_true, y_pred: tf.keras.losses.binary_crossentropy(y_true, y_pred) +
        dice_loss(y_true, y_pred)
)
```

To shorten the training time, the Adam optimizer was chosen, due to its fast convergence. To further improve it, a base learning rate (10^{-4}) was set, with a finer value (10^{-5}) to be updated after the plateau is reached.

7 Output validation

The output will be probabilistic $y_{prob} \in [0,1]^{[H \times W]}$. The mapping will be performed by separating the probabilities by a threshold of 0.5.

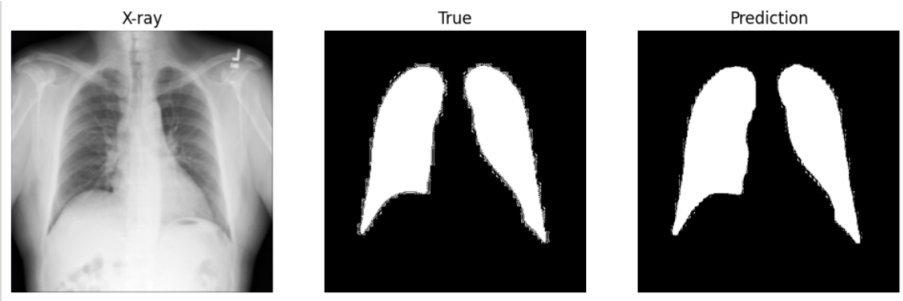
```
    y_pred = (y_prob > 0.5).astype(np.uint8)
```

To assess how good the predictions are, a dice score was rolled and averaged on the prediction set.

```
def dice_score(y_true, y_pred, smooth=1):
    inter = np.sum(y_true * y_pred)
    return (2*inter + smooth) / (np.sum(y_true) + np.sum(y_pred) + smooth)

dice = np.mean([dice_score(y_val[i], y_pred[i]) for i in range(len(y_val))])
```

$$dice \approx 0.971$$

Prediction image compared to the given mask and input:

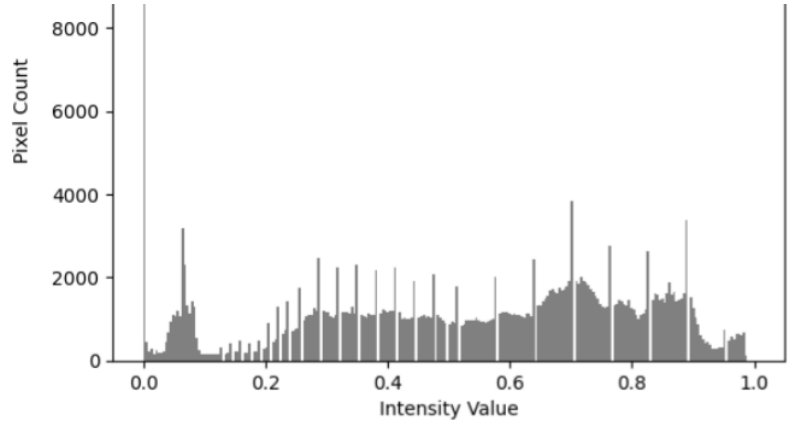


8 Comparison to classical models

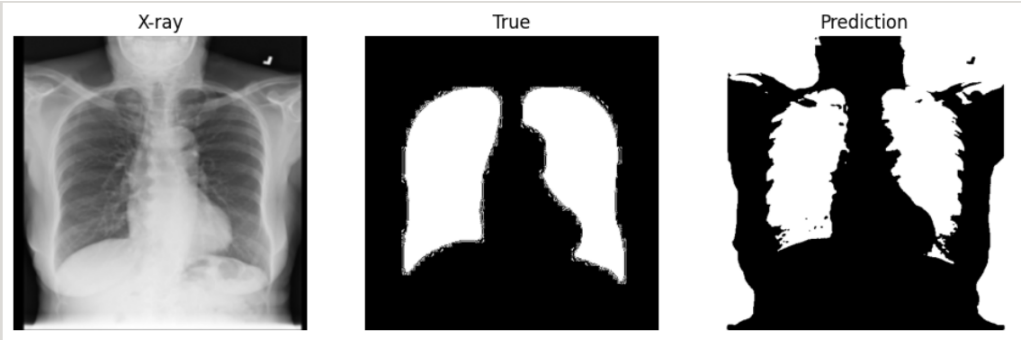
Short introduction to K-means and thresholding:

1. Thresholding:
   — Simple method that requires converting a grayscale image to a binary one by selecting intensity values (thresholds) to separate pixels by category (e.g. foreground/background)

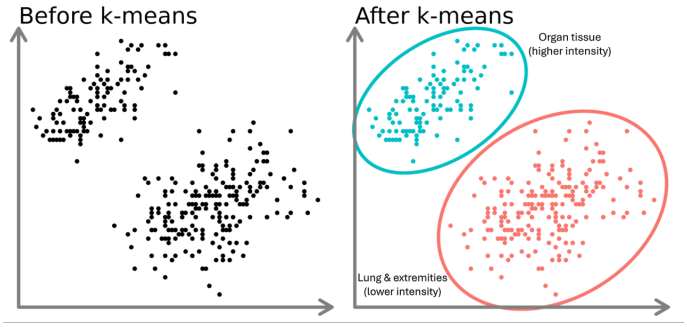
   In this case, a double threshold was necessary to outline the lungs, since they hold an intermediate intensity.



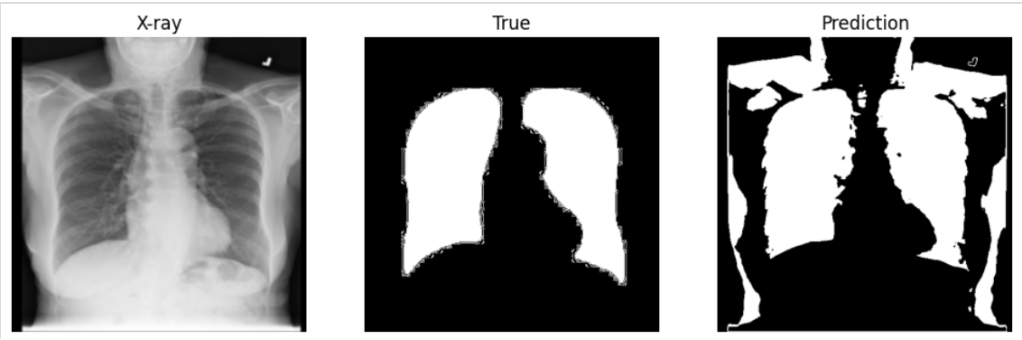
   Due to its inherent lack of surrounding knowledge, the model cannot distinguish non-internal-organ, lighter tissue with the lung area. Thus, although the lungs are well-framed, the model also highlights incorrect areas.



2. K-Means:
   — Unsupervised machine learning method that partitions the image into K distinct regions by grouping into clusters similar pixels, by intensity or color.



The problem with the classical models still stands: lack of spatial awareness. Due to clustering relying solely on pixel intensity, the K-Means model fails at understanding the anatomical context and thus marking as foreground also non-lung area.



Analysis of the results via dice score:

U-Net	Double Threshold	K-Means
0.971	0.603	0.549

9 Conclusion

The U-Net model significantly outperforms classical methods, achieving a Dice score of 0.971 compared to 0.603 (double thresholding) and 0.549 (K-Means), confirming its superior segmentation accuracy.

Classical methods fail because of their lack of spatial and anatomical awareness, leading to incorrect foreground detection, despite significant intensity separation.

Although architecturally and computationally more complex, U-Net provides reliable and consistent segmentation with the anatomy of the human body, justifying its use for medical imaging tasks.